GUIDE 20 · ACCESS

Read the audit log to answer "who changed this, and when"

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=20-read-the-audit-log. This copy was made on 4 November 2026.

Time: 5 minutes. **You'll end up with:** the answer to "who gave this key write access?", "when did that connection's password change?" or "why was that connection deleted?" - and, if you need it, a permanent copy of every change outside QueryAPIGate.

The audit log records **administrative changes**: every create, update and delete of an API key, role, connection or saved query, plus collection moves and renames and loading or removing the example APIs. It doesn't record query runs - those are the run history.

Read it

In the Console: **Observability → Audit log** - newest first, with an action filter and a search box over actor, target and time. **Export** downloads what's shown as JSON.

Over the API (admin key only), filtered by what changed:

```shell
curl 'http://127.0.0.1:5000/api/v1/audit?target=trial-partner' -H 'X-API-Key: demo-key'
```

```json
{
  "items": [
    {"timestamp": "2026-10-04 18:54:22", "actor": "admin", "action": "update_key", "target": "trial-partner",
     "changes": {"allow_writes": {"from": false, "to": true}}},
    {"timestamp": "2026-10-04 18:53:42", "actor": "admin", "action": "create_key", "target": "trial-partner",
     "changes": {"connections": ["shop"], "allow_writes": false, "expires_at": "2026-12-31", ...}}
  ],
  "total": 35,
  "actions": ["create_connection", "create_key", "create_role", "delete_connection", "delete_role",
              "load_examples", "save_query", "update_connection", "update_key", "update_role"],
  "retention": 500
}
```

That answers "who gave this key write access, and when": the admin key, at 18:54:22.

- **An update** shows only what changed, as `{"from": ..., "to": ...}`.
- **A create or delete** shows the whole entry as it was - so a deleted key's grants are still on record.
- **`actor`** is the administrator's name - `admin` for the shared `QUERYAPIGATE_API_KEY`, `cli` for command-line operations such as `queryapigate examples load`. **`via`** says how they signed in: `token` (their own admin token), `break-glass` (the shared key), `cli`, or `open`. Scoped keys can't change anything, so they never appear here.
- **`actions`** lists every action present, handy for building a filter.

Filters

PARAMETER	MATCHES
`action`	one action, e.g. `update_key`, `delete_connection`, `move_query`
`actor`	`admin` or `cli`
`target`	the key, role, connection, query or collection name
`q`	text anywhere in the time, actor or target

```shell
curl 'http://127.0.0.1:5000/api/v1/audit?action=delete_connection' -H 'X-API-Key: demo-key'
curl 'http://127.0.0.1:5000/api/v1/audit?q=2026-10-04' -H 'X-API-Key: demo-key'   # everything changed
that day
```

Secrets are never in it

A connection password is masked: `"********"` in a snapshot, and only `"changed"` in an update - verified:

```json
{"action": "update_connection", "target": "shop", "changes": {"password": "changed"}}
```

API key secrets and hashes are never recorded. The log is safe to export and share with an auditor.

Deleted connections: why, not just when

Deleting a connection breaks every saved query that uses it, so the API requires a reason:

```shell
curl -X DELETE http://127.0.0.1:5000/api/v1/connections/old-reporting -H 'X-API-Key: demo-key'
# {"error": "A reason is required to delete a connection", "code": "reason_required", ...}

curl -X DELETE http://127.0.0.1:5000/api/v1/connections/old-reporting -H 'X-API-Key: demo-key' \
  -H 'Content-Type: application/json' -d '{"reason": "replaced by the warehouse connection"}'
```

The reason is kept in the `delete_connection` entry, and `GET /api/v1/connections/deleted` lists deleted connections with who and why:

```json
{"items": [{"name": "old-reporting", "db": "sqlite", "database": "shop.db", "deleted_at": "2026-10-04
18:54:22",
            "deleted_by": "admin", "reason": "replaced by the warehouse connection"}]}
```

Keeping it longer than 500 entries

The log keeps the newest **500** entries (`retention` in the response); older ones roll off. Two settings:

- `QUERYAPIGATE_AUDIT_LOG_LIMIT=5000` - keep more in the store.
- `QUERYAPIGATE_AUDIT_LOG_EXPORT_FILE=/var/log/queryapigate/audit.jsonl` - **also** append every entry to a file, one JSON object per line, never trimmed:

```text
{"timestamp": "2026-10-04 18:54:22", "actor": "admin", "action": "delete_connection", "target": "old-reporting", "changes": {..., "deleted_reason": "replaced by the warehouse connection"}}
```

The file is the one to keep for compliance: ship it to your log system, or rotate it with `logrotate`. The two writes are independent - a problem writing the file (its directory missing, say) doesn't stop the change from being made or recorded in the store.

Check nobody has edited it

Every entry is chained to the one before it (since 0.25):

```shell
queryapigate audit verify
# Intact: entries 1 to 214, keyed with QUERYAPIGATE_AUDIT_KEY.
queryapigate audit verify --file /var/log/queryapigate/audit.jsonl   # the export file, on its own
```

Set `QUERYAPIGATE_AUDIT_KEY` (`python -c "import secrets; print(secrets.token_hex(32))"`), kept outside the store, so an edit made with only the store can't be re-signed. A break is named - *BROKEN at entry 87 (update_key, 2026-10-12 09:14:02): changed* - and raises the `audit_chain_broken` alert. See the chain.

What it can't tell you

- **Which person** used the shared admin key: everyone who has it is `admin`. Give each person their own admin token (guide 43) and the log names them; any later use of the shared key is then flagged as `break_glass_used`.
- **Changes made outside QueryAPIGate** - restoring a backup, or deleting the newest entries: the chain catches an edit to, or a removal from, the entries it holds, but its end can't know what came after it. A copy in your SIEM does.

Next steps

- Verify who can reach what - the current state, where this is the history.
- Back up and restore the store - the audit log is in the store, so it's in the backup.
- Read this project's threat model.